

Edge Layer Overview

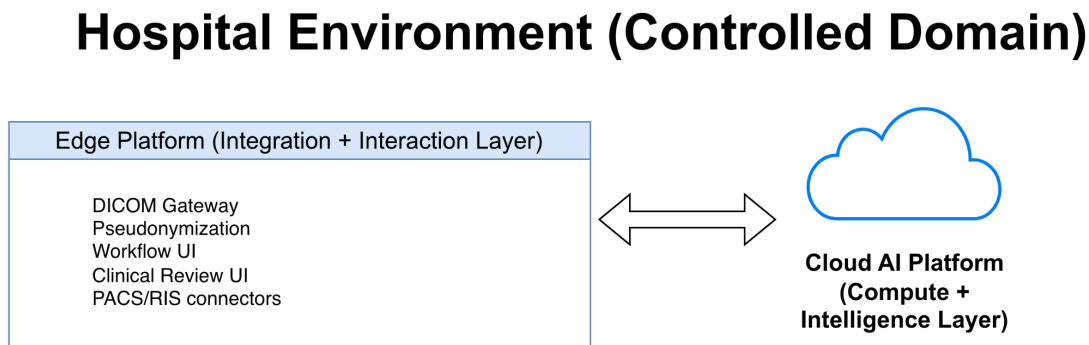
Each business scenario is implemented as a distributed workflow spanning two execution domains: a **hospital-edge clinical execution layer** responsible for data acquisition, governance, and human interaction, and a **cloud-based intelligence layer** responsible for AI processing, model lifecycle management, and scalable computation.

The Edge layer is not optional but an integral part of every scenario involving clinical interaction or regulated data handling.

The Edge layer is not only a data ingestion component — it is the **primary clinical interaction boundary**.

Key Architectural Considerations

A realistic model is:



So the Edge becomes **The Clinical Control Plane** inside the hospital, not just a data pipe. Hospitals typically require:

Data sovereignty

- patient data must stay inside controlled boundary
- even anonymized data may require approval

Clinical latency constraints

- radiologist cannot wait for cloud UI
- review must be local or near-local

PACS integration constraints

- PACS systems are not “API-first”
- integration is often DICOM-native + vendor-specific

Workflow continuity

Clinicians expect one interface for imaging + AI + validation, not 3 separate systems.

Edge Platform as Hospital AI Clinical Gateway has 4 responsibilities:

Data Integration Layer

Connects to:

- PACS
- RIS
- HIS/EHR (sometimes)

Protocols:

- DICOM (C-STORE / C-FIND)
- HL7 v2
- FHIR (in modern hospitals)

[DICOM Standard Overview \(NEMA\)](#)

[DICOM Standard Documentation](#)

[HL7 FHIR Official Specification](#)

[HealthIT.gov FHIR Overview](#)

<https://www.postdicom.com/it/blog/how-pacs-integrates-with-ris-and-ehr-systems>

Privacy & Security Layer

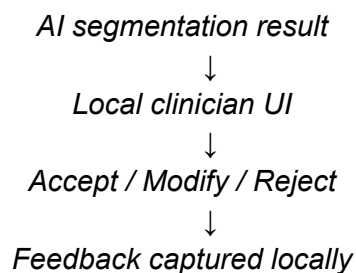
- pseudonymization
- de-identification
- policy enforcement
- consent checks (if applicable)

Clinical Interaction Layer

The edge is also:

- review interface
- validation UI
- annotation tool
- approval workflow console

Example:



Cloud Gateway Layer

- batching
- compression
- secure transfer
- retry logic

ADR: Edge-Centric Clinical Interaction Model

Decision

The platform adopts an edge-centric architecture where clinical interaction, data governance, and integration with hospital systems are implemented within a hospital-deployed edge module, while AI computation and model lifecycle management are centralized in the cloud.

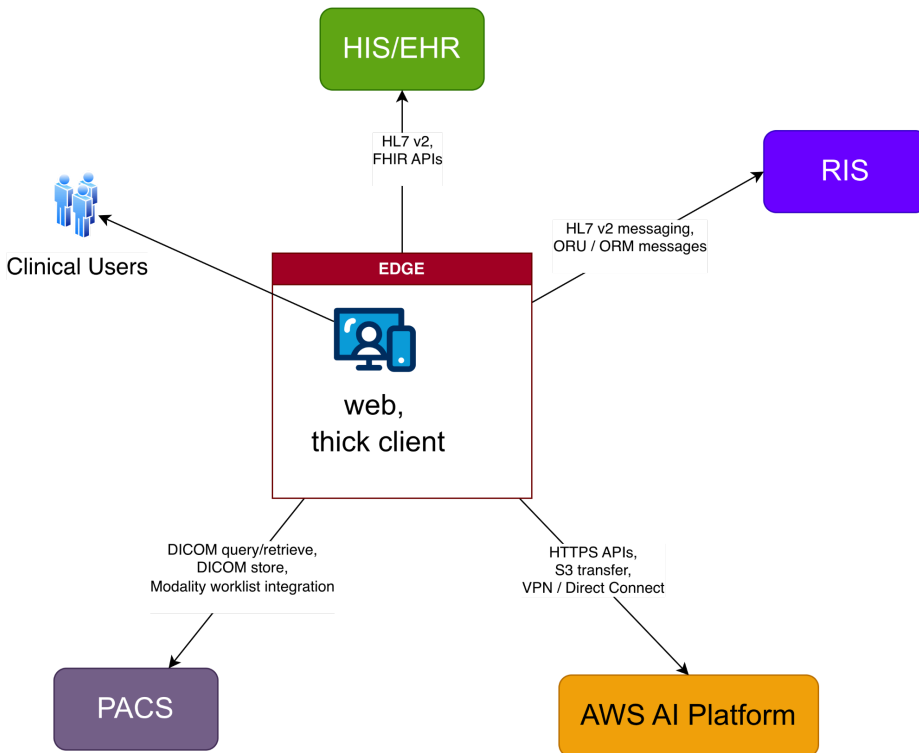
Rationale

- Regulatory constraints (GDPR, medical device compliance)
- PACS/RIS integration limitations
- Latency requirements for clinical workflows
- Clinical trust and usability requirements

Consequences

- Increased complexity in edge deployment
- Stronger hospital IT dependency
- Reduced cloud UI dependency
- Better adoption probability

Integration Scenarios



Edge ↔ PACS Integration

Pattern

- DICOM query/retrieve
- DICOM store
- Modality worklist integration (optional)

Constraints

- vendor-specific PACS implementations
- firewall restrictions
- no direct cloud access

Edge ↔ RIS Integration

Pattern

- HL7 v2 messaging
- ORU / ORM messages

Use

- study scheduling context
- workflow prioritization

Edge ↔ HIS/EHR Integration

Pattern

- HL7 v2 or FHIR APIs

Use

- patient context enrichment
- clinical history retrieval

Edge ↔ AWS AI Platform

Pattern

- asynchronous batch processing
- event-driven sync
- encrypted object transfer

Technologies

- HTTPS APIs
- S3 transfer
- VPN / Direct Connect

6.5 Edge ↔ Clinical Users

Pattern

Local application (web or thick client)

Functions:

- AI result visualization
- 3D model inspection
- annotation tools
- decision capture

Edge Reference Architecture (Logical View)

- Clinical Applications(UI / Visualization)
- Workflow & Orchestration
- Security & Privacy Services
- Integration & Connectivity
- Local Data & Caching Services
- Cloud Synchronization Layer

UI Layer

It must expose clinician-facing functionality. The most simple solution is to implement a Web Application.

Stack

- React
- Angular
- Vue

Backend:

- ASP.NET Core
- FastAPI
- [Node.js](#)
- PACS integration extension

Pros

- easy deployment
- browser-based
- cross-platform

Cons

- large DICOM studies can stress browser rendering, lower local integration

Integration Layer

Responsibilities

- DICOM ingestion
- HL7 processing

- FHIR APIs
- PACS communication
- RIS communication
- EHR communication

Option A — Orthanc-Centric

Components

- Orthanc
- REST APIs
- DICOM plugins

Pros

- open source
- lightweight
- medical imaging focus

Cons

- limited enterprise workflow features

Option B — Mirth Connect + Orthanc

Components:

- Mirth Connect
- Orthanc

Pros

- HL7 expertise
- healthcare standard

Cons

- more components
- Licenses costs

Security Layer

One of the most important. Responsibilities

- RBAC
- audit logs
- pseudonymization
- encryption
- Authentication

Authentication Options

Keycloak

Pros:

- OpenID Connect
- SAML
- MFA

Cons:

- additional operational complexity

Entra ID / Active Directory Federation

Pros:

- integrates with hospital identity

Cons:

- depends on hospital infrastructure

Recommendation: Keycloak, for a vendor-neutral architecture.

Workflow Segmentation

Edge Domain	Study arrives ↓ Anonymize ↓ Policy Validation ↓ Upload Request ↓ ▼
Cloud Platform	Segmentation ↓ Result Packaging ↓ ▼
Edge Domain	Review ↓ Correction ↓ Approval

Option A — Temporal

Pros

- resilient
- stateful
- retries
- excellent hybrid support

Cons

- learning curve

Option B — Camunda

Pros

- BPMN
- business-friendly

Cons

- more process-centric

Option C — Custom Service

Pros

- simple

Cons

- grows into a maintenance problem

Recommendation

For this project Temporal because it aligns well with distributed AI workflows.

Caching & Local Data Layer

Critical for disconnected operation, mostly for

- studies under review
- AI results
- workflow state
- pending uploads

Option A — PostgreSQL + MinIO

PostgreSQL

Stores:

- workflow state
- metadata
- audit records

MinIO

Stores:

- DICOM files
- FBX models
- AI outputs

Pros

- simple
- cloud-like

Option B — NAS + Database

More traditional hospital approach.

Recommendation

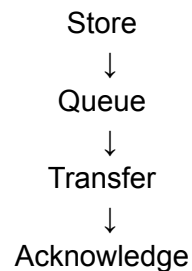
PostgreSQL + MinIO

Synchronization Engine

Responsibilities

- upload studies
- download results
- retry failures
- conflict management

Pattern



Option A — Event Driven

Local event bus.

Examples:

- RabbitMQ
- NATS

Pros

- scalable
- decoupled

Cons

- more infrastructure

Option B — Polling Service

Pros

- simple

Cons

- inefficient

Recommendation

NATS or RabbitMQ

Deployment Platform

Docker Compose

Pros

- simple
- PoC-friendly

Cons

- not enterprise-grade

K3s

Pros

- lightweight Kubernetes
- edge-oriented

Cons

- requires Kubernetes knowledge

OpenShift

Pros

- enterprise

Cons

- expensive and heavy

Recommendation

K3s

Recommended Edge Technology Stack

Capability	Technology
------------	------------

UI	React + ASP.NET Core
DICOM	Orthanc
HL7/FHIR	Mirth Connect
Authentication	Keycloak
Workflow Engine	Temporal
Metadata Store	PostgreSQL
Object Storage	MinIO
Event Bus	NATS
Sync Engine	Custom Service
Deployment	K3s

Architectural Observation

What is emerging is not a "connector".

The Edge is effectively a **micro-platform** that resembles a miniature cloud environment deployed inside the hospital.

The Edge Platform should expose cloud-native capabilities locally, enabling the hospital environment to operate as an autonomous clinical execution domain while leveraging cloud services for AI computation and model lifecycle management.

That statement can guide every technology decision you make afterward.